

SESSION - 07

Introduction to HTL



Table of Contents

Inheritance

Working with HTL 3

Exercise 1 7

Task- Render Basic Page Content 7

Task – Modularize the contentpage component 9

INTRODUCTION TO HTL

Working with HTL

HTL (HTML Template Language) is a templating language developed by Adobe. It is the recommended language to use for developing components using Adobe Experience Manager. The best practice is to use HTML instead of JSP. The AEM Sites reference site, We.Retail, is built using HTL.

An HTL template defines an HTML output stream by specifying the presentation logic and the values to be dynamically inserted into the stream based on some background business logic. HTL differs from other templating systems in four main ways:

1. **HTL is HTML5:** A template created in HTL is a valid HTML5 file. All HTL-specific syntax is expressed within a data attribute or within HTML text. Any HTL file opened as HTML in an editor will automatically benefit from features such as auto-completion and syntax highlighting, which are provided by the editor for regular HTML.

```
<div class="HTL-example">
    <header data-sly-include="header.html"></header>
    <h1 data-sly-test="${properties.title}">${properties.title}</h1>
    <section data-sly-use-navigation="Navigation">
        <h1>
            ${navigation.breadcrumb}
        </h1>
    </section>
    <ul data-sly-list-child="${resource.listChildren}">
        <li>${child.name}</li>
    </ul>
</div>

```

2. **Separation of concerns (Web designer versus web developer):** The expressiveness of the HTL markup language is purposely limited so that only relatively simple presentation logic can be embedded in the actual markup. All complex business logic must be placed in an external helper class. The HTL's Use API defines the structure of the external helper.
3. **Secure by default:** HTL automatically filters and escapes all text being output to the presentation layer to prevent cross-site-scripting vulnerabilities.
4. Compatible with JSP or ESP.

HTL - Two Main Goals:

Goal 1: Simplified development ("Beautiful markup")

- ✓ Less code
- ✓ Cleaner code
- ✓ More intuitive code
- ✓ More efficient

Goal 2: Security

- ✓ Cross-site scripting protection
- ✓ Built-In
- ✓ Automatic
- ✓ Context-sensitive

Comparing HTL to JSP

HTL (less code, more secure)

```
<a href="${properties.link || '#'}" title="${properties.jcr:title}">
   ${properties.jcr:description}
</a>
```

JSP (more code, less secure)

```
<a href="<%= xssAPI.getValidHref(properties.get("link", "#")) %>" <%
   String title = properties.get("jcr:title", "");
   if (title.length() > 0) {
      %>title="<%= xssAPI.encodeForHTMLAttr(title) %>"<%
   } %>>
   <%= xssAPI.encodeForHTML(properties.get("jcr:description", "")) %>
</a>
```

Both code blocks above achieve the same functional result.

HTL Syntax

There are two different types of syntaxes:

1. **HTL Block Statements -** To define structural elements within the template, HTL employs the HTML data attribute, which is HTML5 attribute syntax purposely intended for custom use by third-party applications. All HTL-specific attributes are prefixed with **`data-sly-`**.

2. **HTL Expressions** - HTL expressions are delimited by characters ${ and }. At runtime, these expressions are evaluated and their value is injected into the outgoing HTML stream. They can occur within the HTML text nodes or within the attribute values. In other words, you include HTL expressions within HTML tags.

The following is a list of a few of the basic HTL statements, expressions, and tags.

Comments (HTL comments are HTML comments with additional syntax. They are delimited like shown here.**)**
Example: `<!--/* An HTL Comment */-->`

Expressions
Examples: `${true}`, `${properties.text}`

URI Manipulation
Example: `${'example.com/path/page.html' @ scheme='http'}` (Resulting output: http://example.com/path/page.html)

Enumerable objects
Examples: `pageProperties`, `properties`, `inheritedPageProperties`

HTL Block Statements

- ✓ use: `<div data-sly-use.nav="navigation.js">${nav.foo}</div>`
- ✓ unwrap: `data-sly-unwrap`
- ✓ list: `<dl data-sly-list="${currentPage.listChildren}">`
- ✓ `data-sly-include, data-sly-repeat, etc.`

Special HTML tags
Example: `<sly>`

Expressions contain two types:

1. Literals

- Boolean: `${true} ${false}`
- Strings: `${'foo'} ${"answer"}`
  - NOTE: All of the global.jsp objects are available automatically!
- Positive integers: `${42}`
- Arrays: `${[42, true, 'Hello World']}`

2. Variables

`${myVar}`
Variables are accessed as: `${properties.myVar}`

Page-Rendering Scripts

When you create a component, a rendering script is also created by default. When you view a page, the output is displayed from the rendering script.

Adobe recommends you use HTL as the rendering script. Apart from using the HTL tags, you just need to change the extension of the script from .jsp to .html. As you will see in later exercises (starting in Module 10), you will be doing this task quite often in this course.

Use APIs to Display Basic Page Content

This section is an introduction to rendering the content from the repository, which is a similar concept to querying and displaying data from a database. In the repository, the nodes define the structure and the properties hold the data. To render content on any page, you need to render the data from those properties. Initially, start with the basic properties associated with every page.

Enumerable Objects and Java-Backed Objects: Summary

- Enumerable objects can be iterated-through using data-sly-list.
  - properties
  - pageProperties
  - inheritedPageProperties
- Java-backed objects
  - component – componentContext
  - currentDesign
  - currentPage
  - currentSession
  - request - response
  - resource – resourceDesign – resourcePage
  - wcmmode
  - etcetera...

Here are some of the ways that you can access the JCR in Adobe Experience Manager:

The **currentPage** object (Java-backed)
The currentPage object is an instance of the page (see AEM API) class, which provides some methods to access content. For example: `${currentPage.Title}`

The **properties** object (Enumerable)
The properties object is an instance of the ValueMap (see Sling API) class and contains all properties of the current resource. For example: `<p> Title : ${properties.jcr:title}</p>`

The **currentNode** object (Java-backed)
The currentNode object is an instance of the Node (see JCR API) class, which provides access to content using the getProperty() method. For example: `${currentNode.Name}`

Modularize Page Components

It is important to modularize a component into multiple scripts and include them at runtime—promoting component or script reuse. You use the HTL include tags to support modularization. There are different ways in which you can include a file to the script.

For example, you can include a file at runtime using HTL as: `<div data-sly-include="myScript.html"/>`

HTL Resources

Adobe Documentation and Tutorials:
https://docs.adobe.com/docs/en/htl/overview.html and https://docs.adobe.com/docs/en/htl/docs/getting-started.html

HTL Spec

The HTML Template Language Specification GitHub Repository is available as a reference for all HTL attributes, behavior and syntax:
https://github.com/Adobe-Marketing-Cloud/htl-spec

https://github.com/Adobe-Marketing-Cloud/htl-spec/blob/master/SPECIFICATION.md

Exercise - I

Task – Render Basic Page Content

At this point, all of the pages in the We.Train website are identical. Now we will begin to use HTL to render the content residing in properties.

1. **Using CRXDE Lite, navigate to the /apps/training/components/structure/contentpage node.**
2. **Open contentpage.html.**
3. **Replace the existing code in CRXDE Lite with the code from the Exercise Files and save your changes.**

   NOTE: The code is provided as part of the Exercise_Files under **\Module 7 Introduction to HTL\Task - Render Basic Page Content**. Copy and paste the code from the exercise files to CRXDE Lite. Do not copy from this exercise book. The following is for illustration purposes only.

contentpage.html

```
<!DOCTYPE html!>
<!--/* A simple HTL script */-->
<html>
       <head>
       <meta charset="utf-8"/>
    </head>
    <body>
       <h1>Hello World!!</h1>
        <h3>Sling PropertiesObject</h3>
                    <p>Page Title : ${properties.jcr:title}</p>

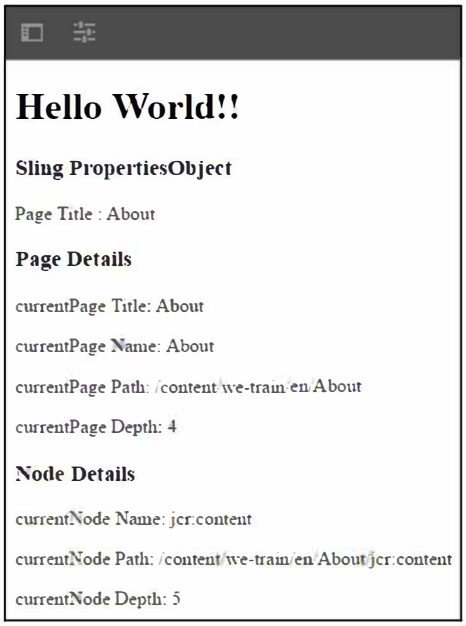
              <h3>Page Details</h3>
                    <p>currentPage Title: ${currentPage.Title}</p>
                    <p>currentPage Name: ${currentPage.Name}</p>
                    <p>currentPage Path: ${currentPage.Path}</p>
                    <p>currentPage Depth: ${currentPage.Depth}</p>

        <h3> Node Details </h3>
                    <p>currentNode Name: ${currentNode.Name}</p>
                    <p>currentNode Path: ${currentNode.Path}</p>
                    <p>currentNode Depth: ${currentNode.Depth}</p>
    </body>
</html>
```

4. **Using the Sites Console, navigate to Sites > We.Train > English and open the page for editing (HINT: Use the Edit button).**



5. **Examine the rendered content.**
6. **Open the About child page of English in Edit mode.**



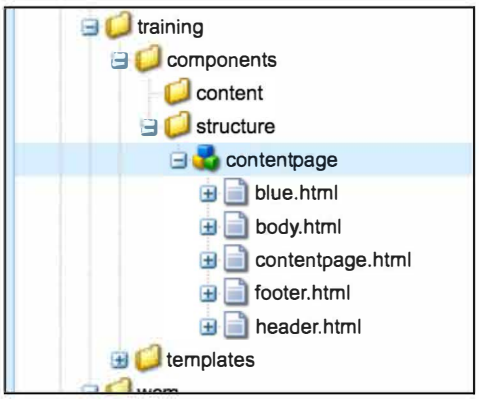
7. **Examine the rendered content. Notice the differences as compared to the English page. Each page is now unique because we are rendering (for each page) the properties associated with that page.**

What we have just done consists of the basics for page rendering. As we have mentioned before, in the repository, the nodes provide the structure and the properties hold the data. By rendering the property values associated with the page, we have rendered the page content.

Task – Modularize the contentpage Component

Modularization of components enables reuse of common structures and reduces code redundancy.

1. **Using CRXDE Lite, right-click /apps/training/components/structure/contentpage and select Create...> Create File.**
2. **Enter body.html for the file name. Save your changes.**
3. **Repeat steps 1 and 2 to create header.html and footer.html.**



4. **Examine the code for body.html, header.html, and footer.html.**

body.html

```
<div class="container we-Container--main">
    <div class="root responsivegrid">
        <div class="aem-Grid aem-Grid--12 aem-Grid--default--12  ">
            <div class="header aem-GridColumn aem-GridColumn--default--12" data-sly-
include="header.html"></div>
            <div class="hero-image image parbase aem-GridColumn aem-GridColumn--default-
-12" style="padding-top: 100px;">
                <div>Hero Component</div>
                <div class="responsivegrid aem-GridColumn aem-GridColumn--default--12">
                    <div class="aem-GridColumn aem-GridColumn--default--12">
                        <div class="row">
      <div>Breadcrumb</div>
                            <div class="we-Header">Title Component</div>
                            <div>Responsive Content Area</div>
                        </div>
                    </div>
                    <form class="page__print">
                        <input value="Print Friendly" type="submit" />
                    </form>
                    <div class="footer aem-GridColumn aem-GridColumn--default--12" data-
sly-include="footer.html"></div>
                </div>
            </div>
        </div>
    </div>
</div>
```

header.html

```
<div class="navbar navbar-inverse navbar-fixed-top hidden-xs">
    <div class="container-fluid">
        <nav style="color: white;" >Language Navigation</nav>
        <ul class="nav navbar-nav navbar-right" style="color: white;" >
            <sly>Toolbar</sly>
        </ul>
    </div>
</div>
<div >Site Navigation</div>
```

footer.html

```
<footer class="we-Footer width-full">
    <div class="container">
        <div class="row">
            <div class="row we-Footer-section we-Footer-section--sub">
                <div class="we-Footer-section-item">
                    <div class="we-Logo we-Logo--big">
                        we.<strong>train</strong>
                    </div>
                    <div class="we-Footer-nav">
                        <h2>Footer Toolbar</h2>
                    </div>
                </div>
            </div>
            <div class="row we-Footer-section we-Footer-section--sub">
                i18n Coded Content
            </div>
            <div class="row">
                <div class="col-md-12">
                    <div class="text-center">
                        <a href="#top" class="btn btn-primary">Back to the top</a>
                    </div>
                </div>
            </div>
        </div>
    </div>
</footer>
```

5. **Using the code from the Exercise_Files, paste the code for each of the three scripts into the three empty HTML files that you created.**

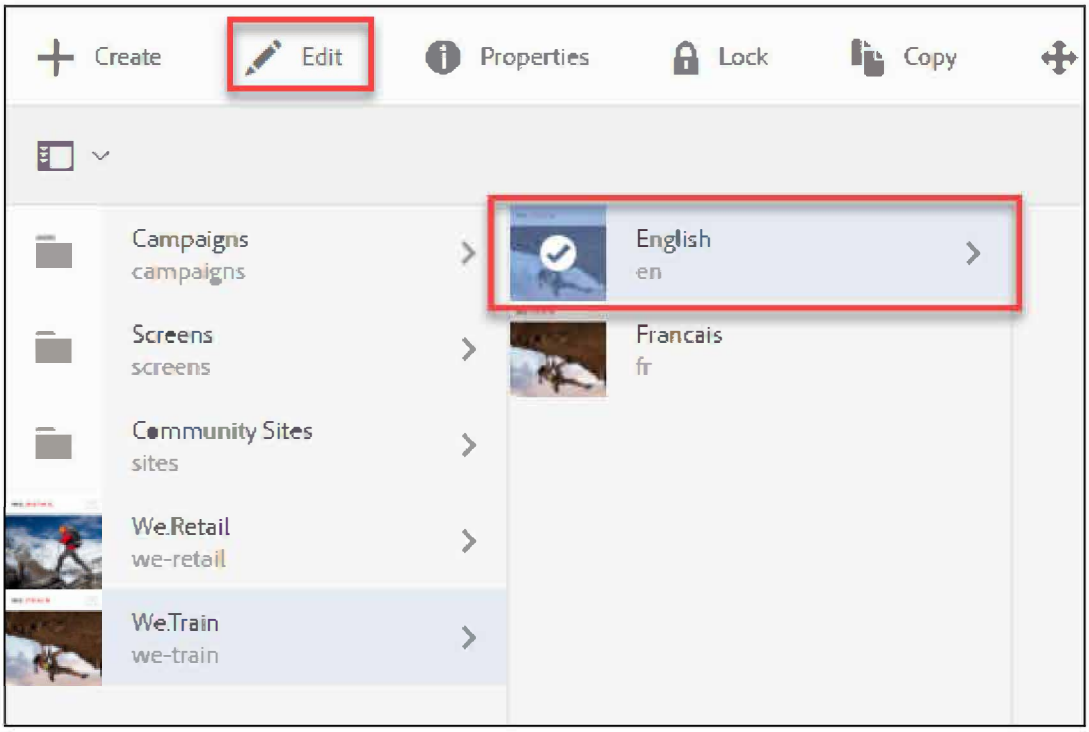
   NOTE: The code is provided as part of the Exercise. Copy and paste the code from the exercise files to CRXDE Lite. Do not copy from this exercise book.

6. **Also, modify the contents of contentpage.html using the content from the Exercise(same directory).**

contentpage.html

```
<!DOCTYPE html!>
<!--/* A simple HTL script */-->
<html>
 <head>
   <title>${properties.jcr:title}</title>
  </head>
  <body>
    <div data-sly-include="body.html"></div>
  </body>
</html>
```

7. **Using the Sites Console, open Sites > We.Train > English.**



ADOBE COPYRIGHT PROTECTED

It should look something like this:



Notice that there are placeholders for components and other elements that we will be implementing throughout the course. There is no design applied yet, so the class definitions that we referenced in the code are not active yet. We will define and apply the design later in the course.